

II2520 - Einführung in Deep Learning

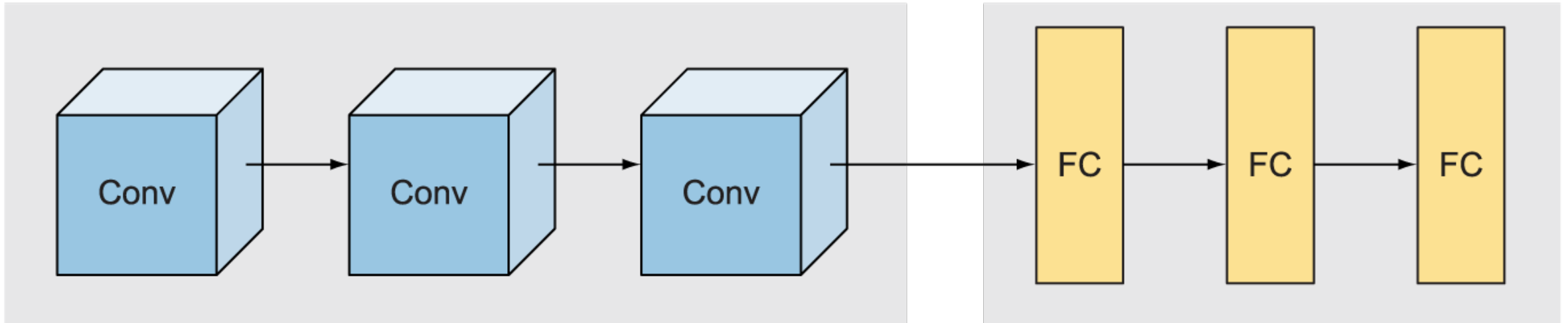
Vorlesung 7: Recurrent Neural Networks

Rückblick

CNN-Entwurfsmuster

Feature extraction

Classification



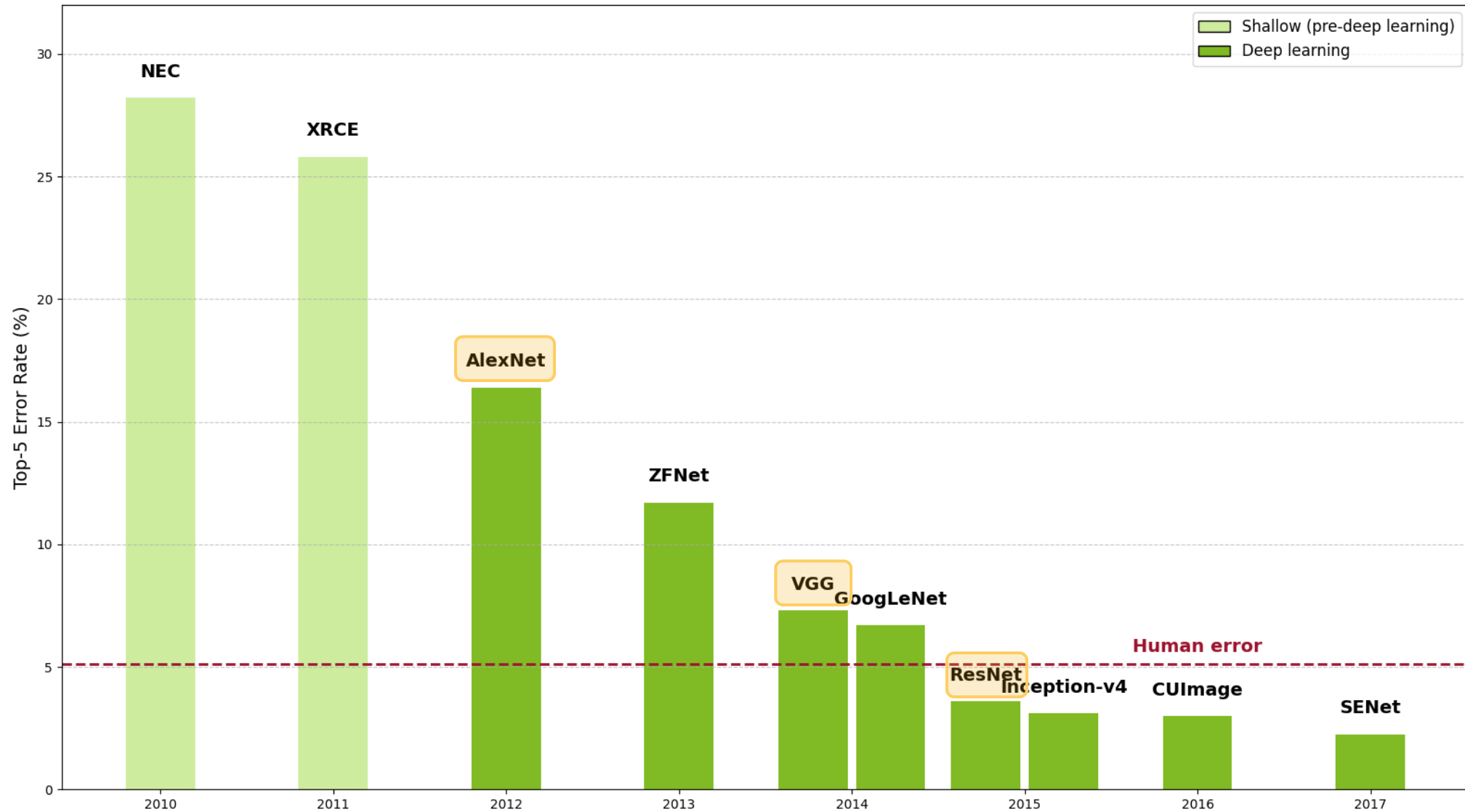
Allgemein gilt:

1. Feature Extractor (Backbone)
2. Classification (Head)

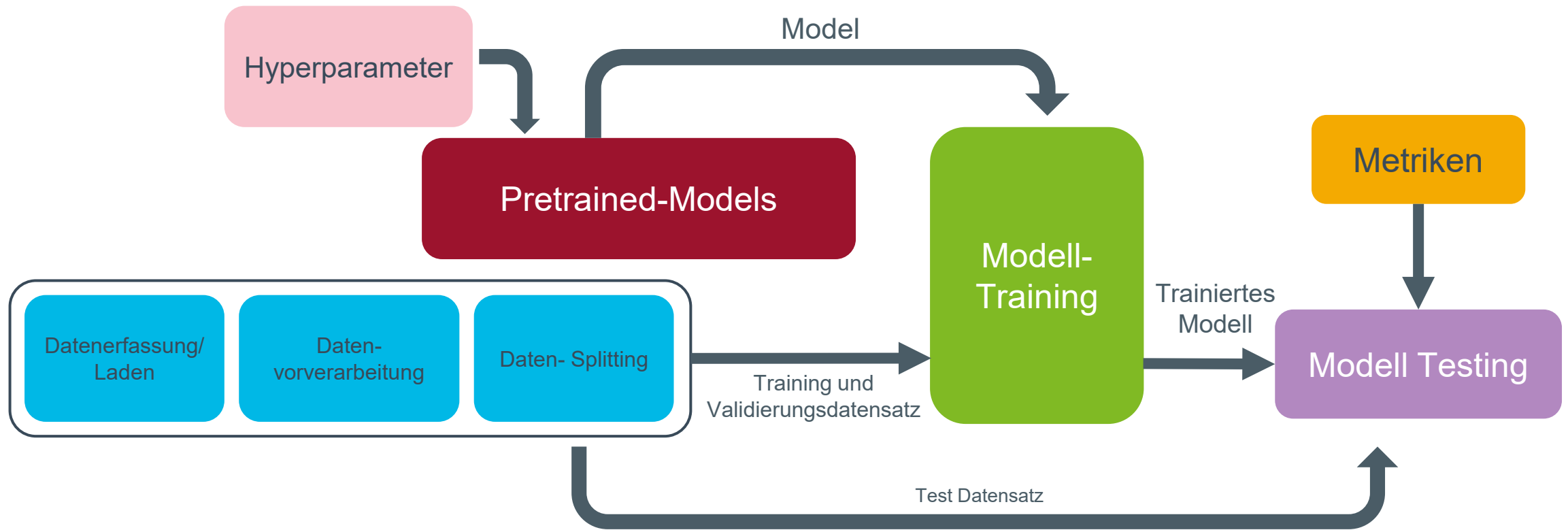
 Kann auch für andere Aufgaben angepasst werden

Bild-Quelle: Mohammed Elgendy, *Deep Learning for Vision Systems*

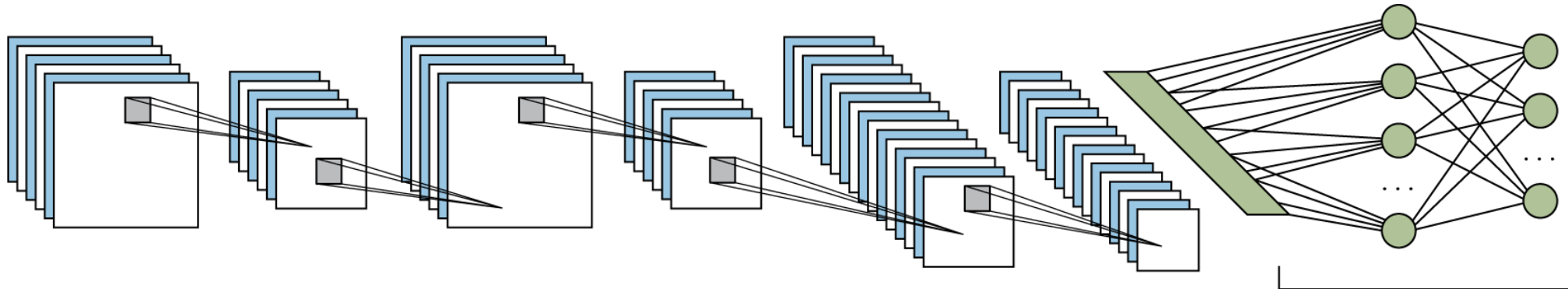
ILSVRC



Deep Learning Pipeline



Transfer Learning



Scenario #1: You have a small dataset that is similar to the source dataset.

Scenario #2: You have a large dataset that is similar to the source dataset.

Scenario #3: You have a small dataset that is different from the source dataset.

Scenario #4: You have a large dataset that is different from the source dataset.

Bild-Quelle: Mohammed Elgendy, *Deep Learning for Vision Systems*

Inhalt

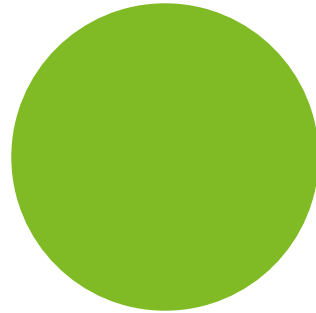


Inhalt dieser Vorlesung

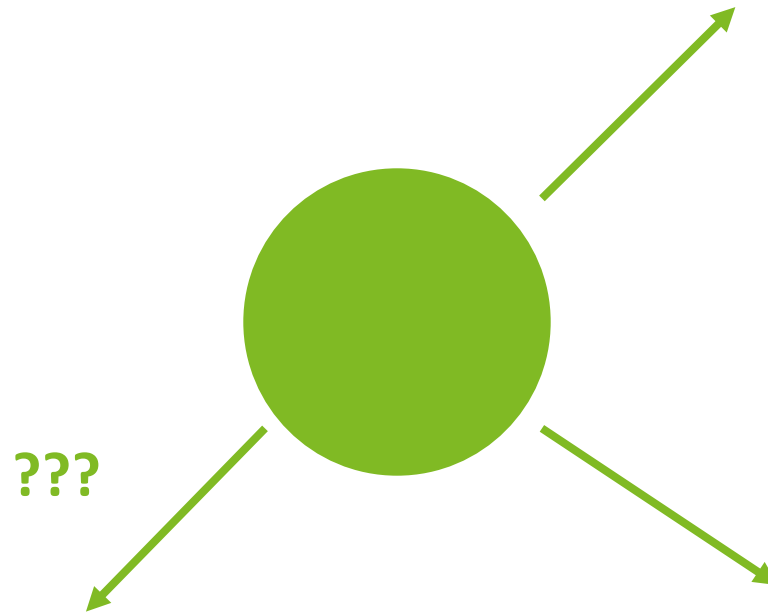


- Besprechung von Evaluation
- Sequenzmodellierung
- Sequenzdaten
- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory (LSTM)

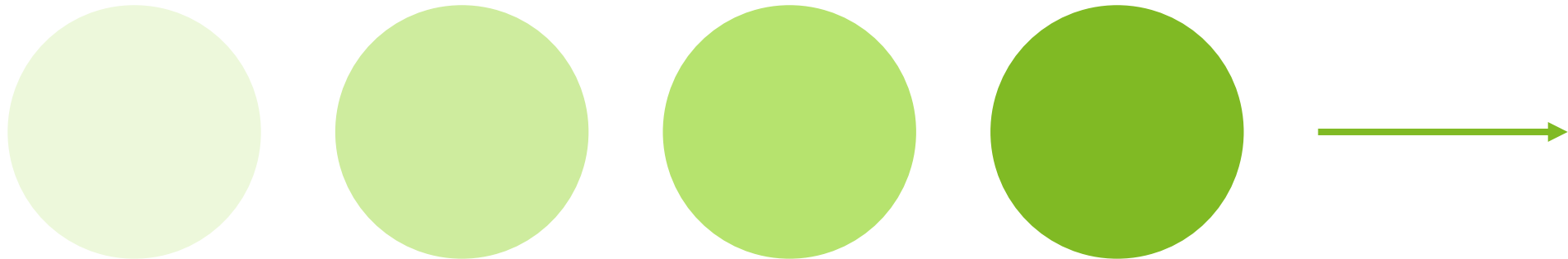
**Kann man anhand des Bildes eines Balls vorhersagen,
wohin er als Nächstes fliegen wird?**



Kann man anhand des Bildes eines Balls vorhersagen, wohin er als Nächstes fliegen wird?



Kann man anhand des Bildes eines Balls vorhersagen, wohin er als Nächstes fliegen wird?



Sequenzmodellierung

Sequenzdaten

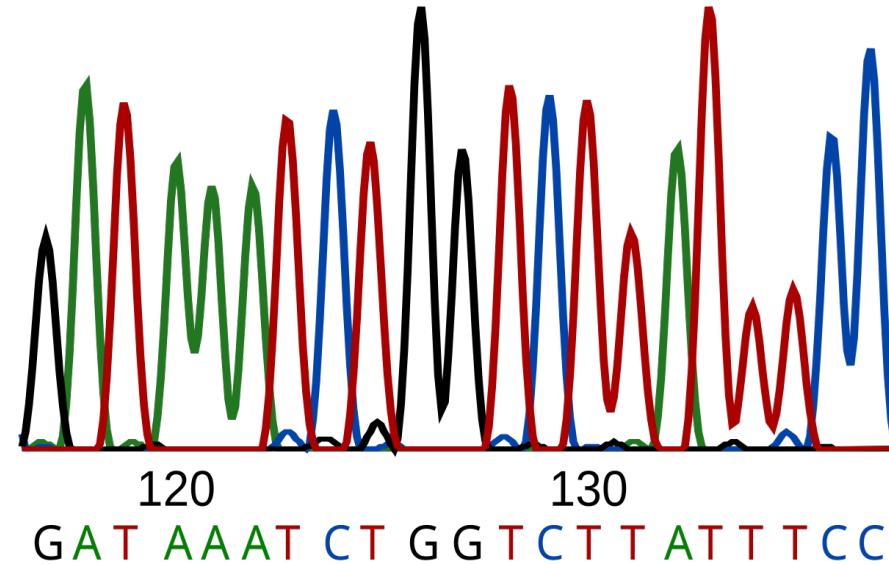
Die Welt ist voll von dynamischen Objekten:

Wir müssen uns Datensequenzen ansehen, um die Veränderungen von einer Momentaufnahme zur nächsten zu verstehen und Vorhersagen zu treffen ("Trajektorie" in den folgenden Bildern)

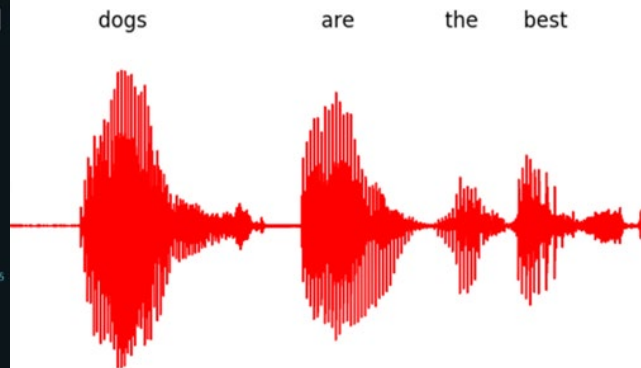


Bild-Quelle: <https://arxiv.org/pdf/1811.02146>

Sequenzdaten

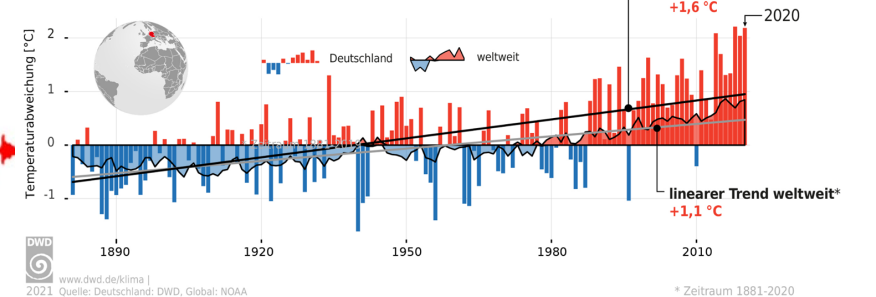


Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.



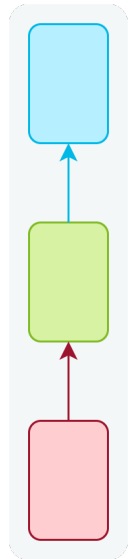
Erwärmungstrend in Deutschland stärker als weltweit

Abgebildet sind die **positiven** und **negativen** Abweichungen der Lufttemperatur vom vieljährigen Mittelwert 1961 - 1990 für Deutschland und weltweit

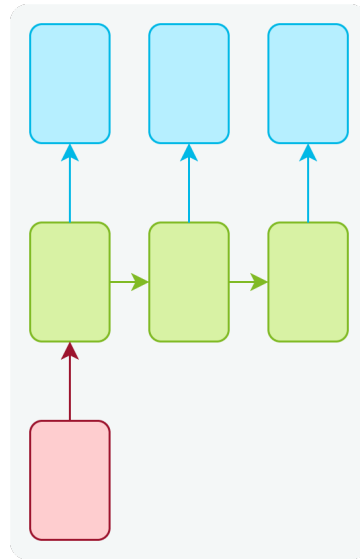


Anwendungen

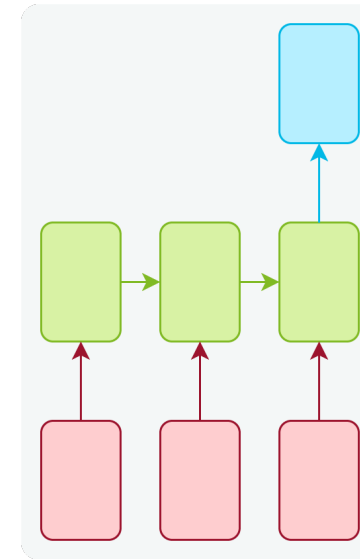
In vielen praktischen Anwendungen enthalten die Eingabe- oder Ausgabebereiche Sequenzen.



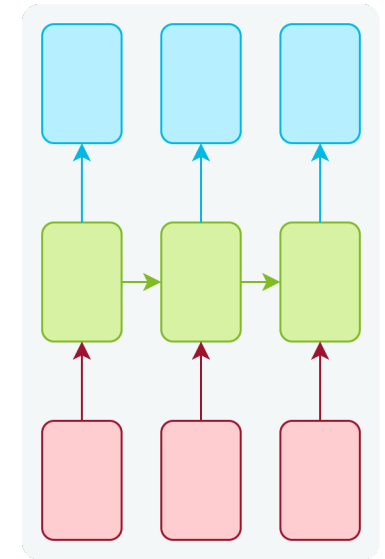
One to One
**Binäre
Klassifikation**



One to Many
Image Captioning

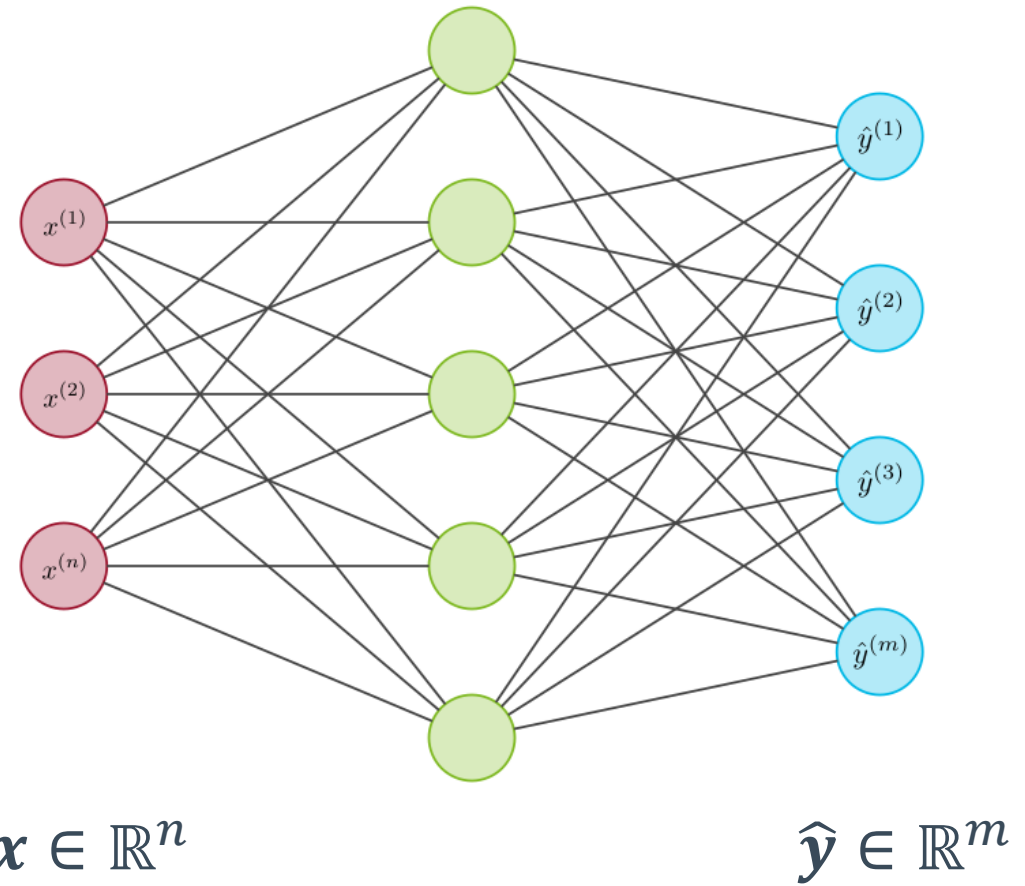
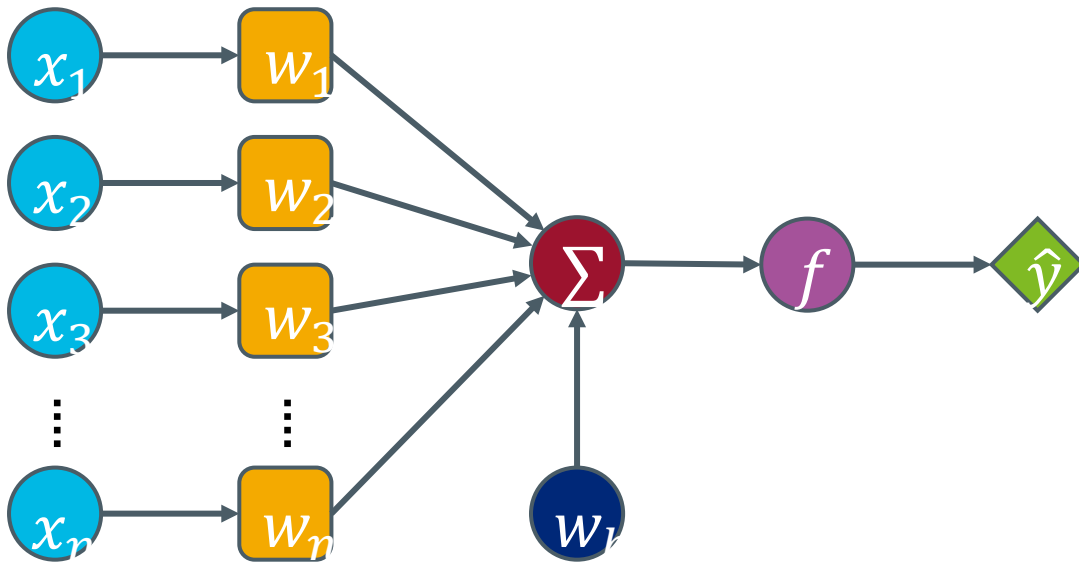


Many to One
**Stimmungs
Klassifikation**

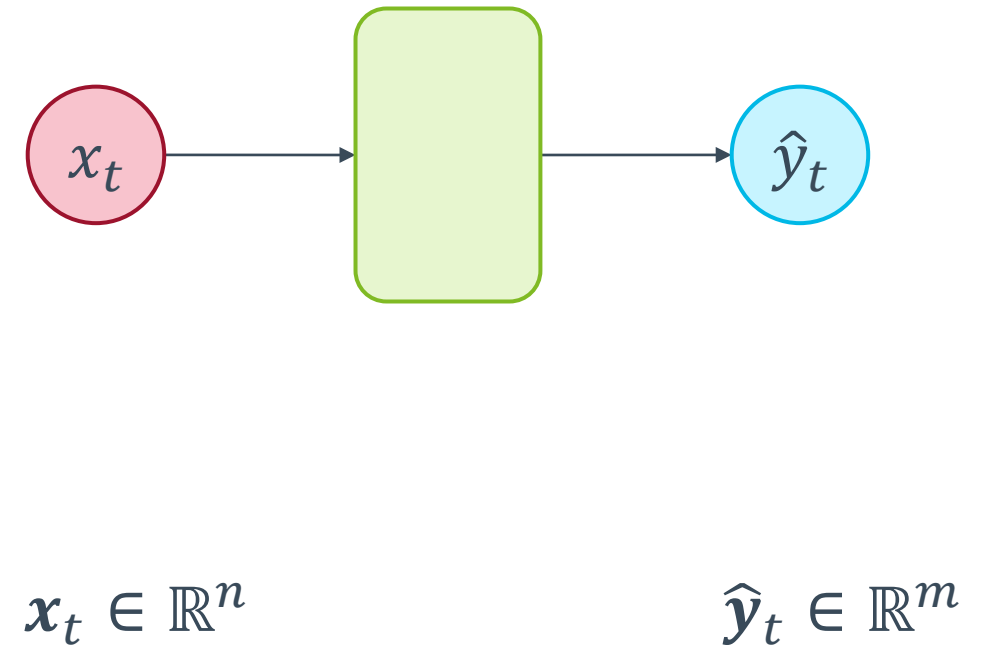
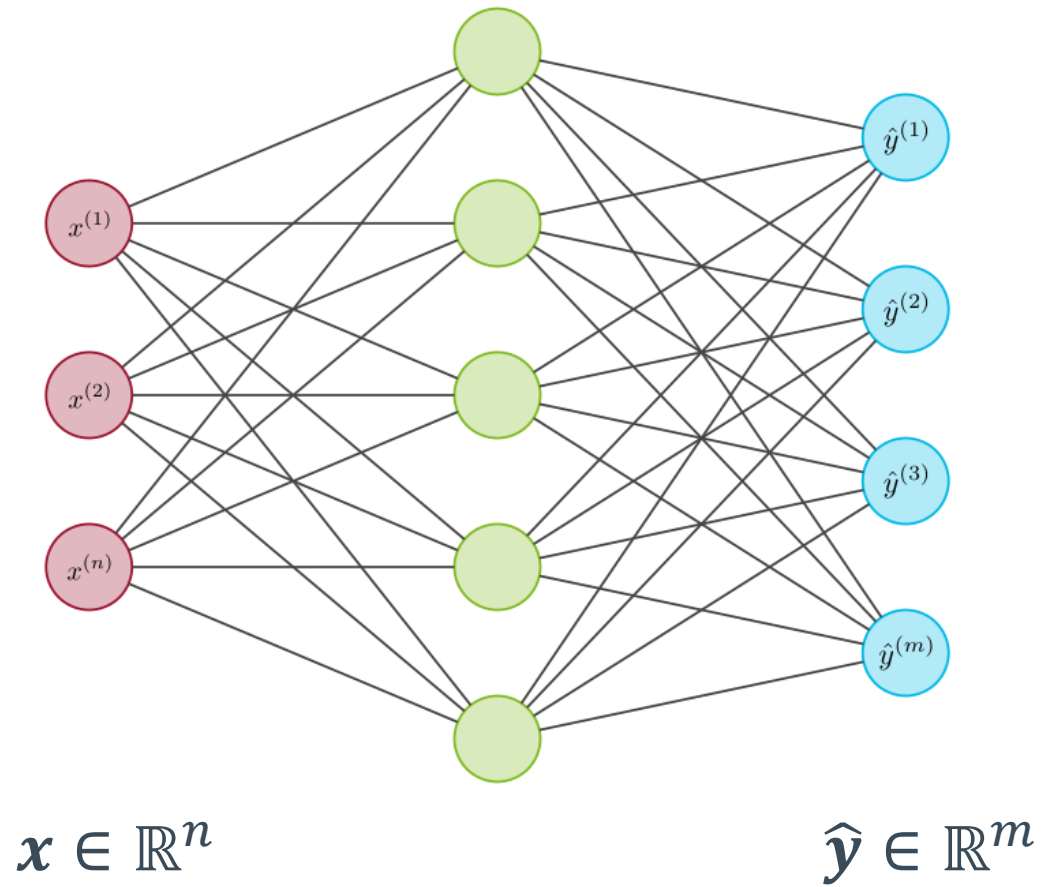


Many to Many
**Maschinelle
Übersetzung**

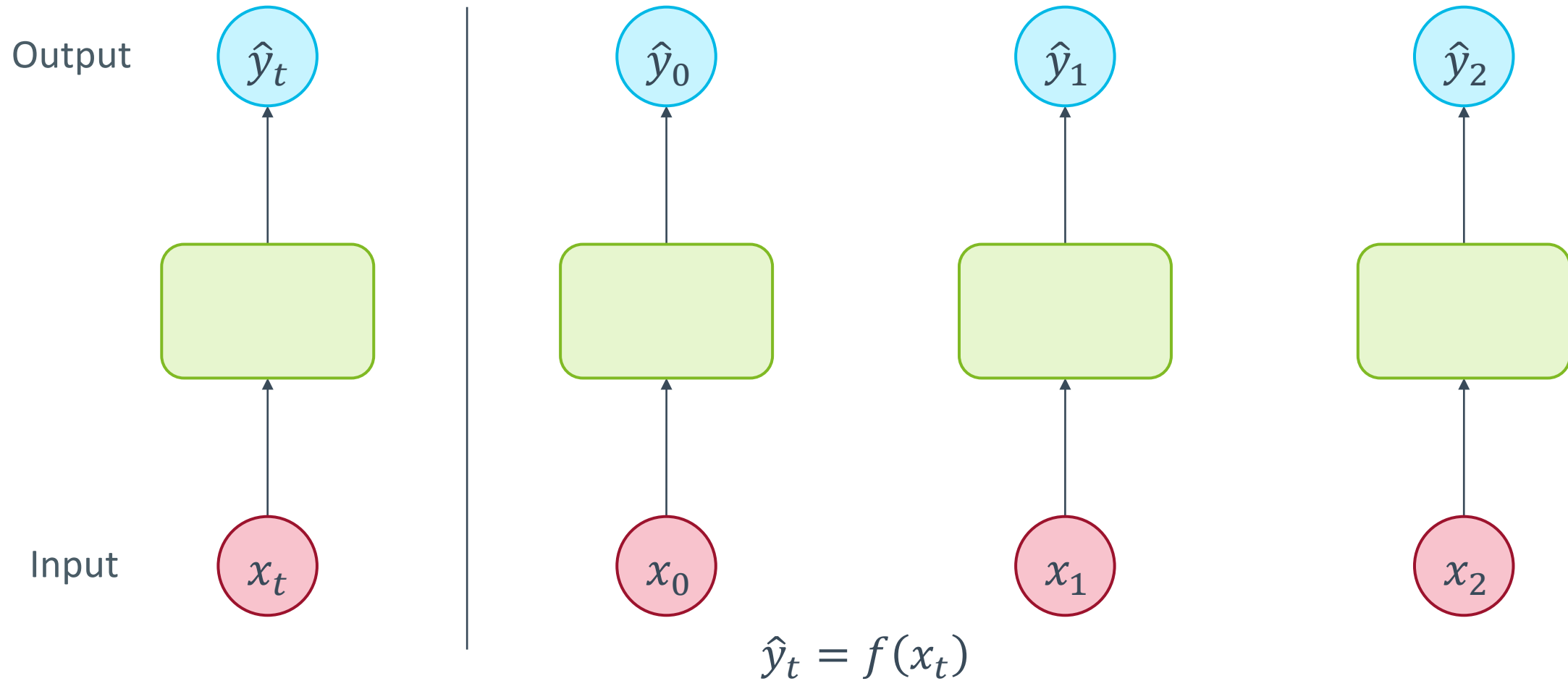
Recap: Perceptron



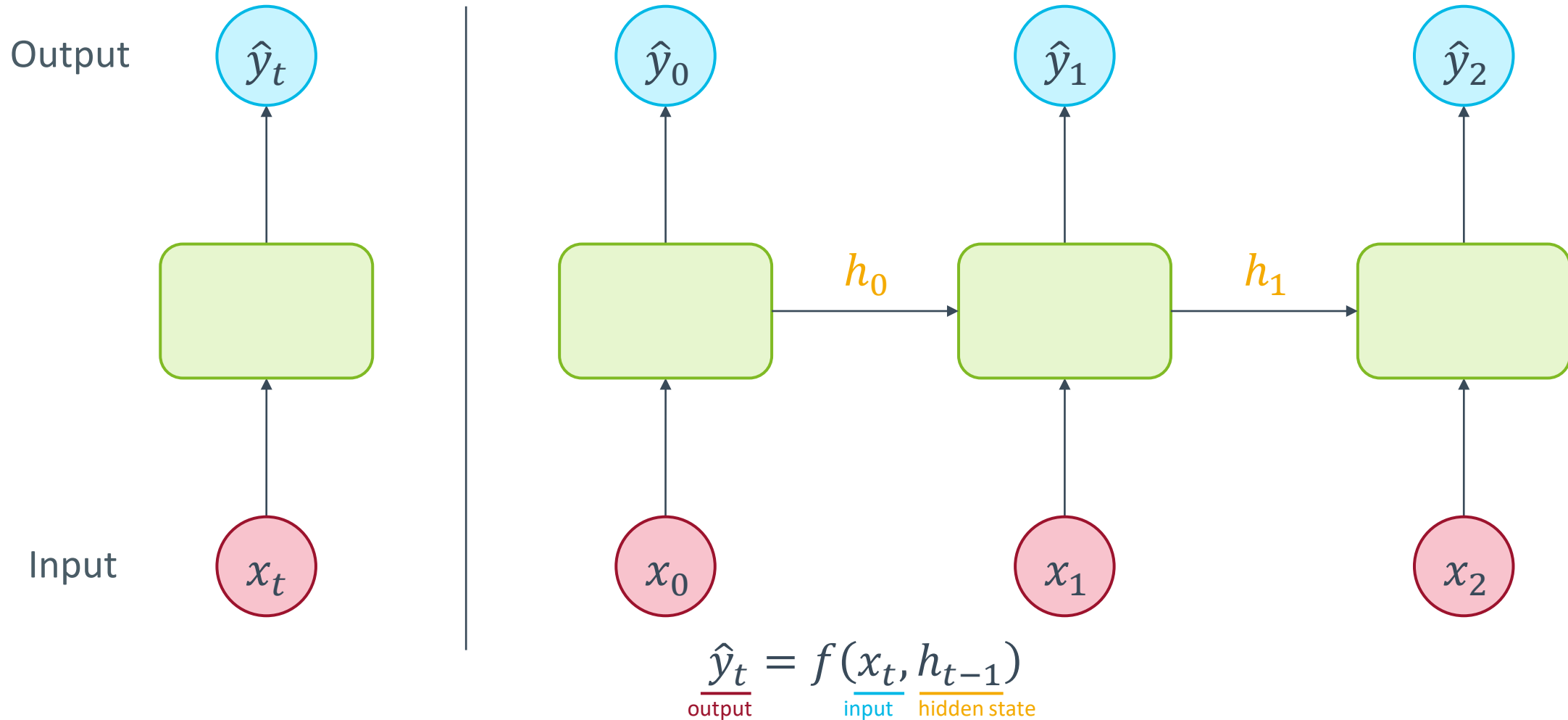
Recap: Perceptron



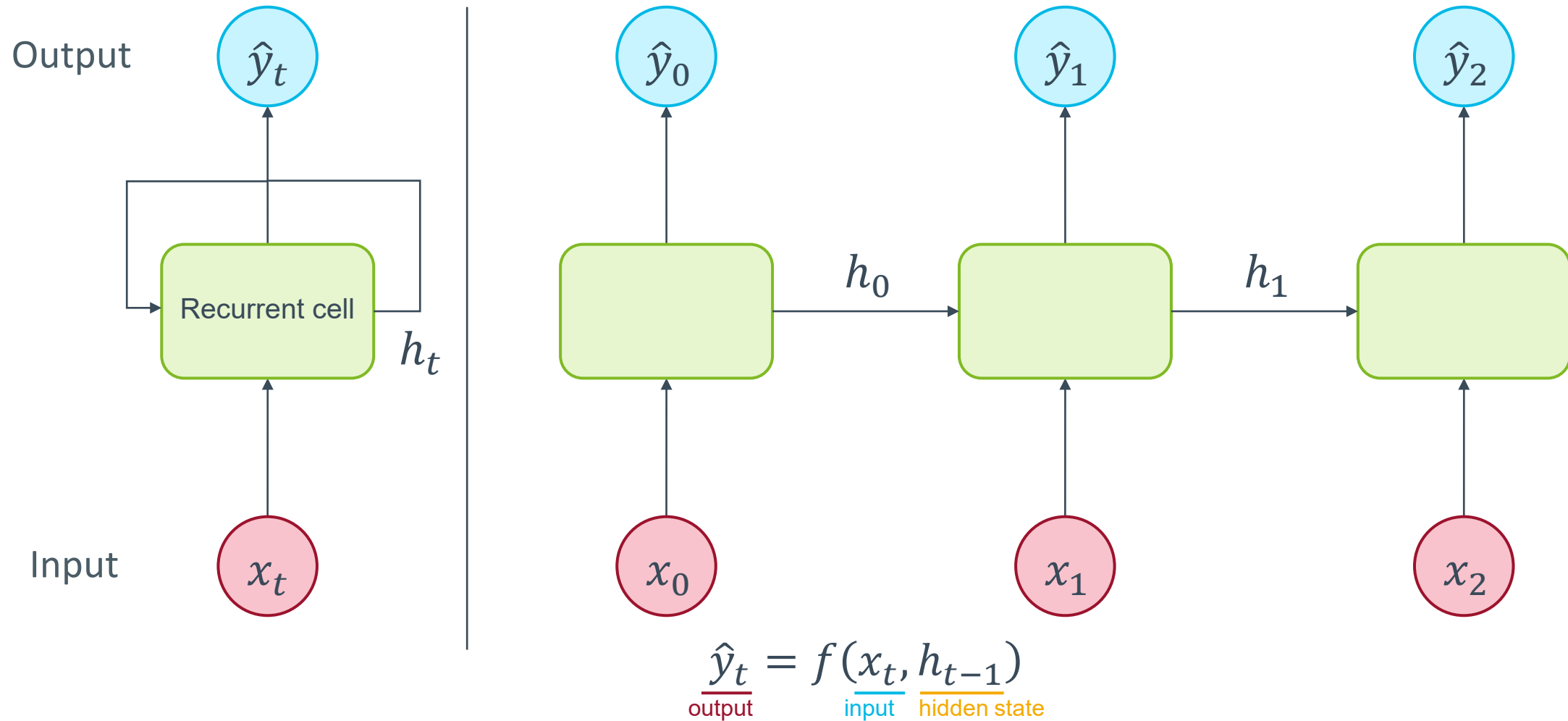
Perceptron: Individuelle Zeitschritte



Neuronen mit Rekursion

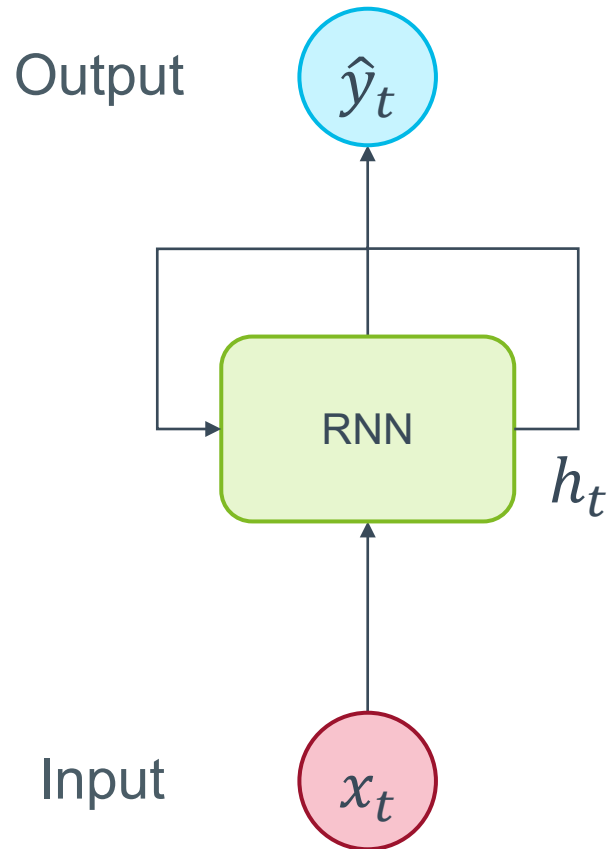


Neuronen mit Rekursion



Recurrent Neural Networks (RNN)

Recurrent Neural Networks (RNN)



Anwendung von Rekursion bei jedem Zeitschritt zur Verarbeitung einer Sequenz

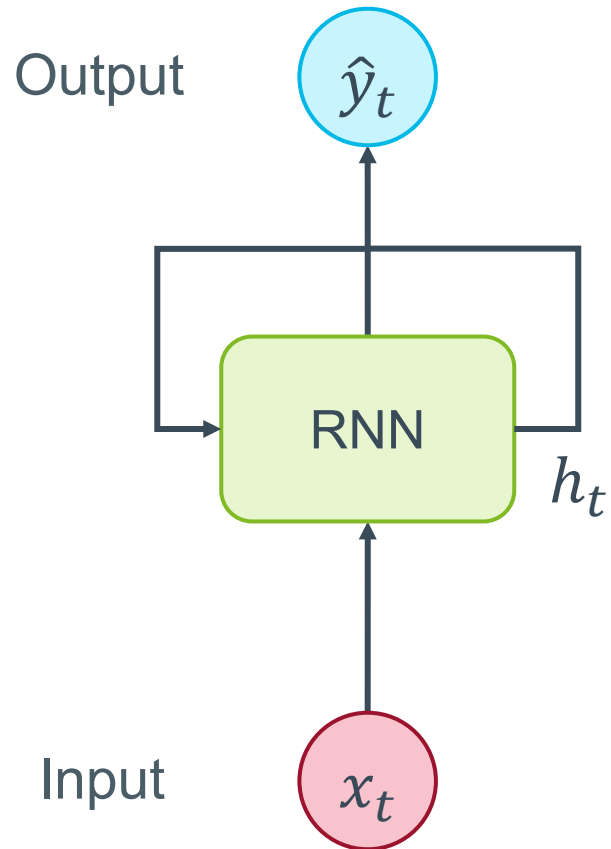
$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

Zustand Funktion Mit Gewichten Input Alter Zustand

Hinweis: dieselbe Funktion mit denselben Parametern wird bei **jedem** Zeitschritt benutzt

RNNs haben einen **Zustand** h_t , welcher bei **jedem Zeitschritt** aktualisiert wird, während eine Sequenz verarbeitet wird

RNN: Update der Parameter



Output Vector

$$\hat{y}_t = \mathbf{W}_{hy}^T h_t$$

Update Hidden State

$$h_t = \tanh(\mathbf{W}_{hh}^T h_{t-1} + \mathbf{W}_{xh}^T x_t)$$

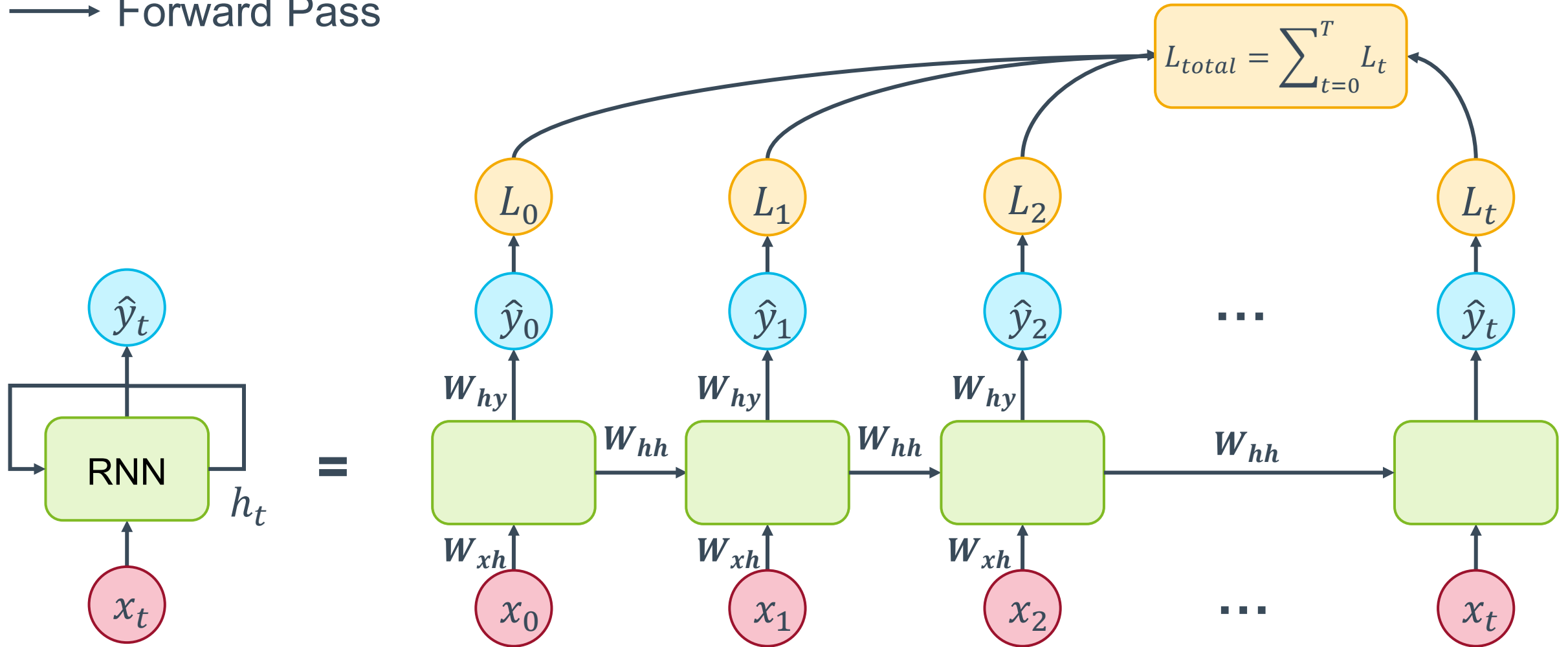
Input Vector

$$x_t$$

RNNs: Computational Graph

Benutze **die gleichen Gewichte** für jeden Zeitschritt

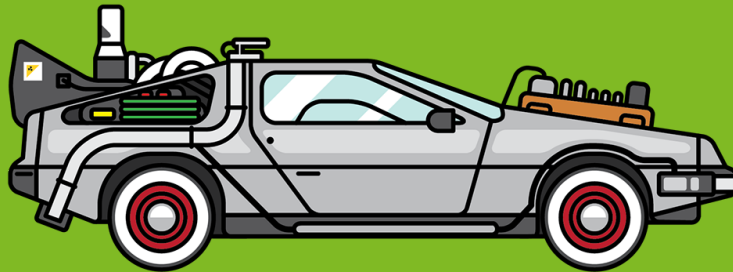
→ Forward Pass



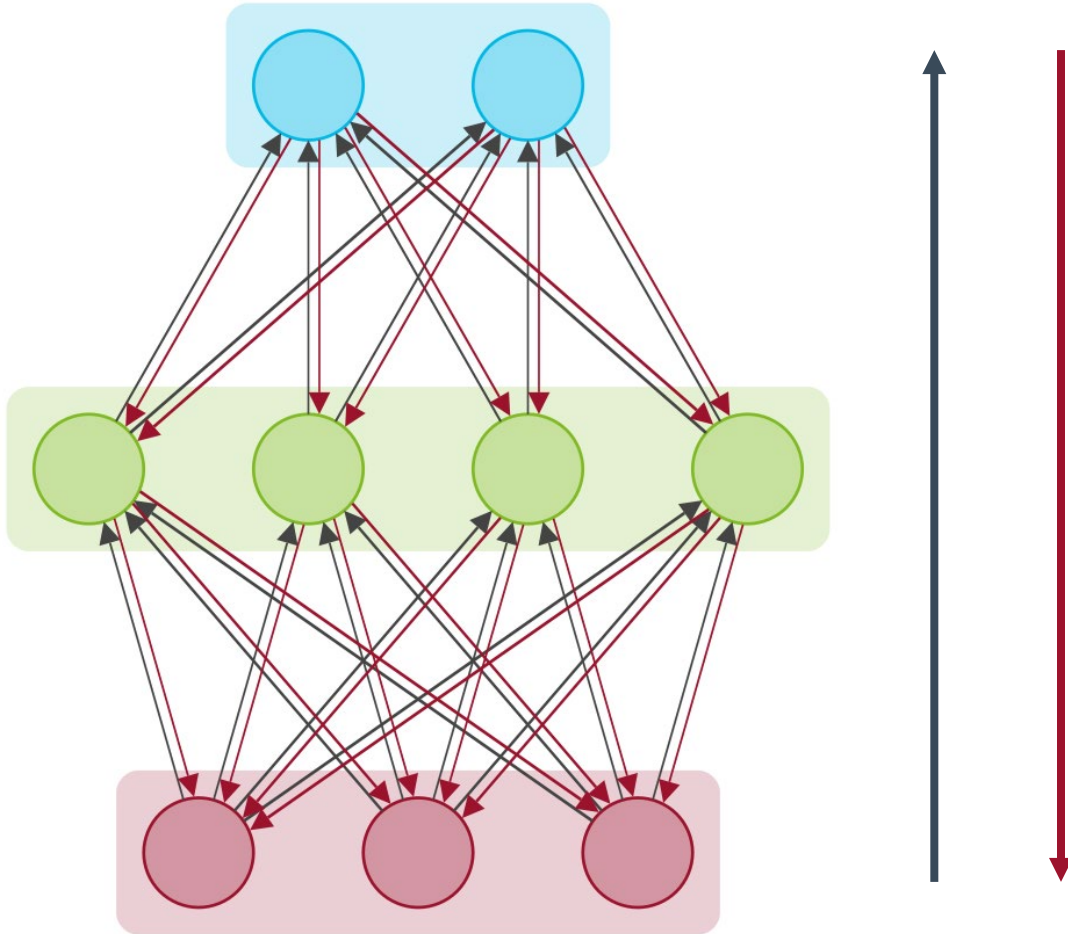
RNN: Beispiel Pytorch

```
1 class SimpleRNN(nn.Module):
2     def __init__(self, input_size, hidden_size, num_layers, output_size):
3         super(SimpleRNN, self).__init__()
4         self.hidden_size = hidden_size
5         self.num_layers = num_layers
6
7         # RNN Layer
8         self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
9
10        # Fully connected layer
11        self.fc = nn.Linear(hidden_size, output_size)
12
13    def forward(self, x):
14        # Initialisiere hidden state mit Nullen
15        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
16
17        # Forward propagation durch RNN
18        out, _ = self.rnn(x, h0)
19
20        # Dekodiere den hidden state der letzten Zeitschritte
21        out = self.fc(out[:, -1, :])
22        return out
```

Backpropagation Through Time (BPTT)



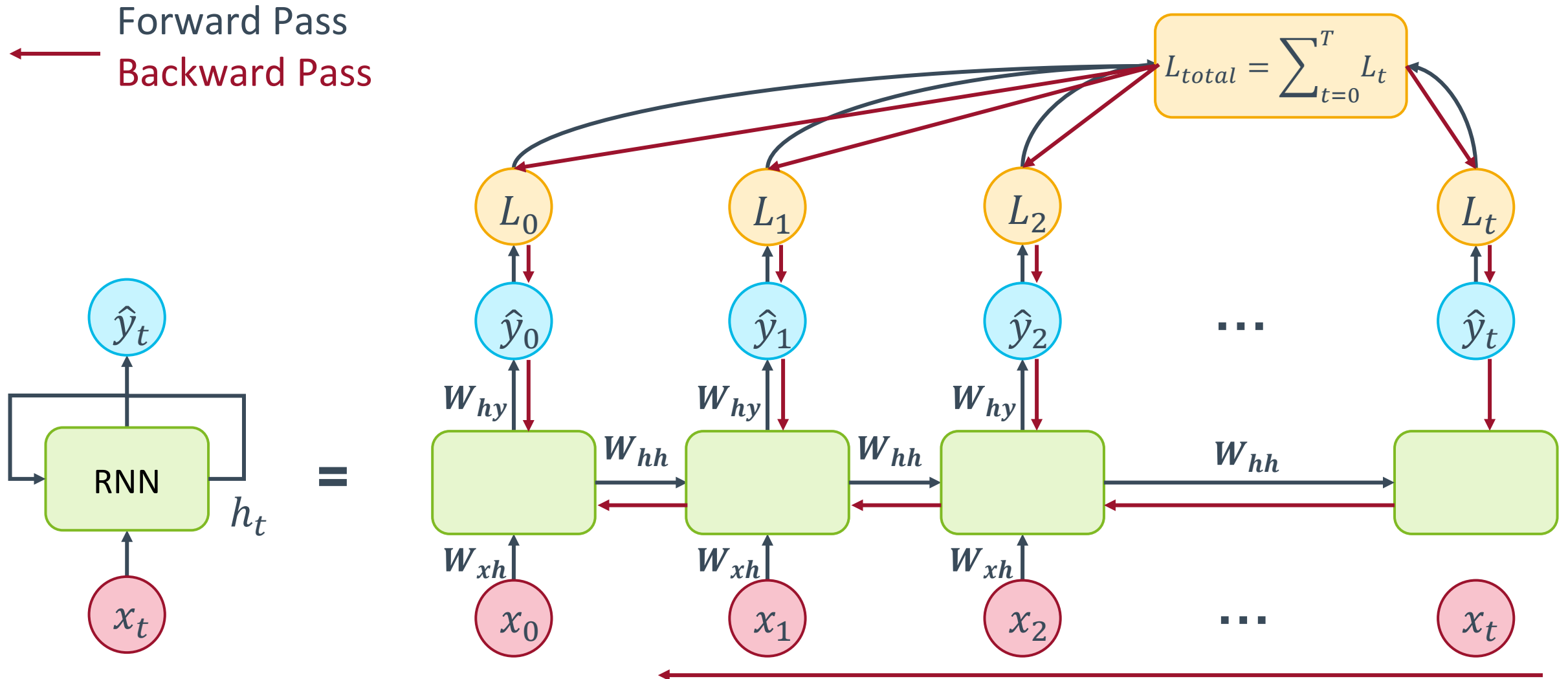
Recap: Backpropagation



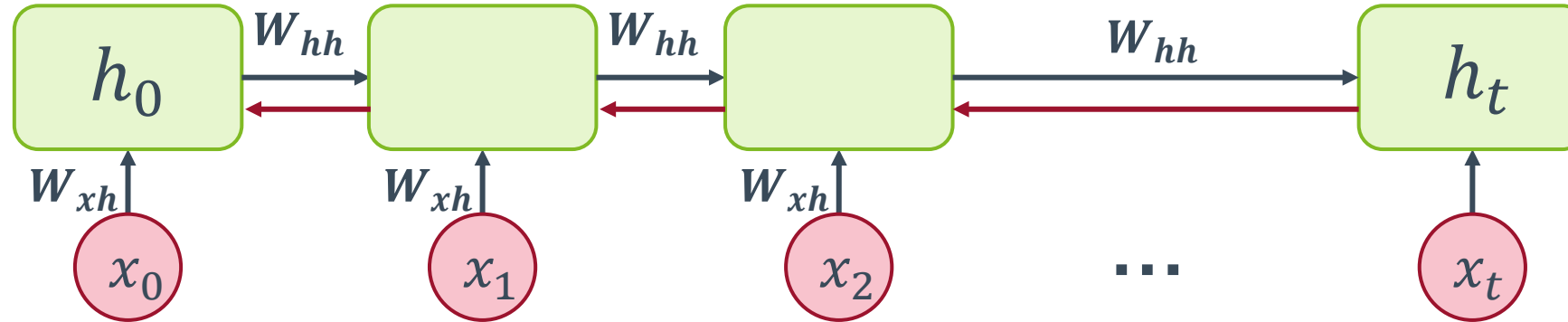
Backpropagation:

1. Berechne die Gradienten in Bezug auf die Loss-Funktion für jeden Parameter
2. Update die Parameter so, dass das Loss minimiert wird

RNNs: Backpropagation Through Time



RNNs: Backpropagation Through Time



Die Berechnung des Gradienten in Bezug auf h_0 erfordert **mehrfache Multiplikation von W_{hh}** und zusätzlich **wiederholte Gradientenberechnungen!**

Viele Werte > 1 :
Explodierende Gradienten

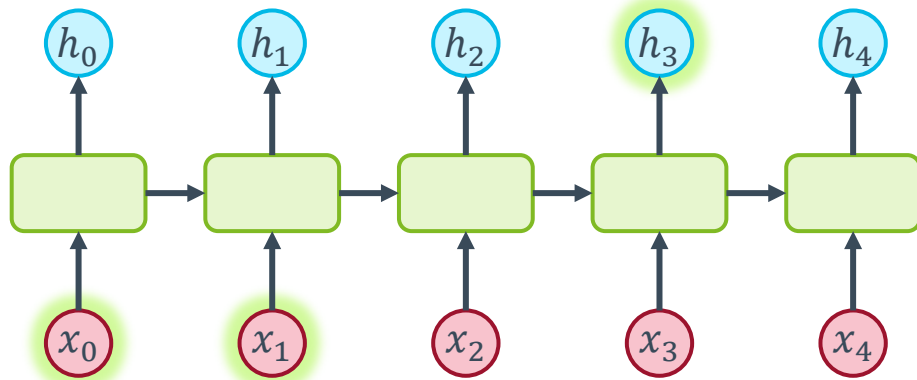
Gradient Clipping zur
Beschneidung von großen
Zahlen

Viele Werte < 1 :
Verschwindende Gradienten

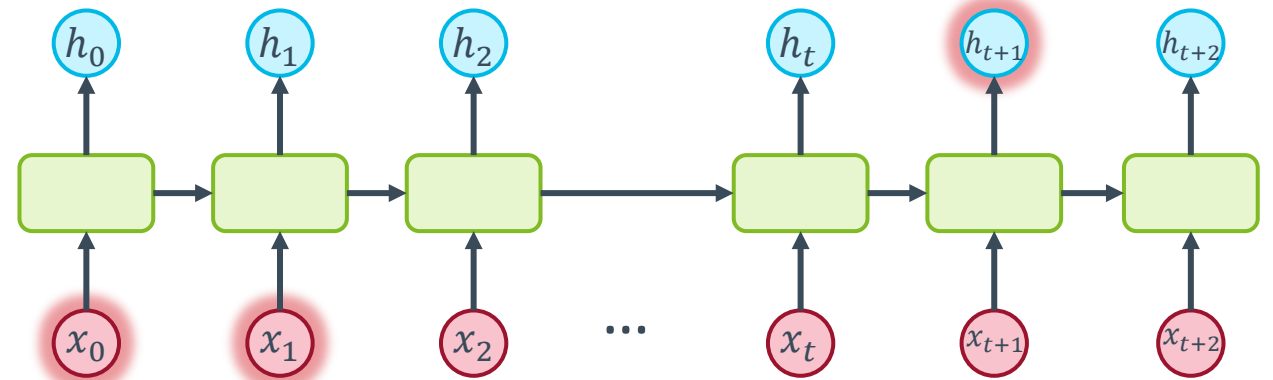
1. Aktivierungsfunktion
2. Gewichte initialisieren
3. Netzwerkarchitektur

Langfristige Abhängigkeiten

- **Hauptproblem von RNN:** Obwohl sie theoretisch in der Lage sein sollten, zum Zeitpunkt t Informationen über Eingaben zu behalten, die viele Zeitschritte zuvor gesehen wurden, ist es in der Praxis unmöglich, solche langfristigen Abhängigkeiten zu lernen.
- Dies ist auf das **Problem des verschwindenden Gradienten** zurückzuführen.
- Wenn die Lücke größer wird, können die RNNs nicht mehr lernen, die Informationen zu verknüpfen.



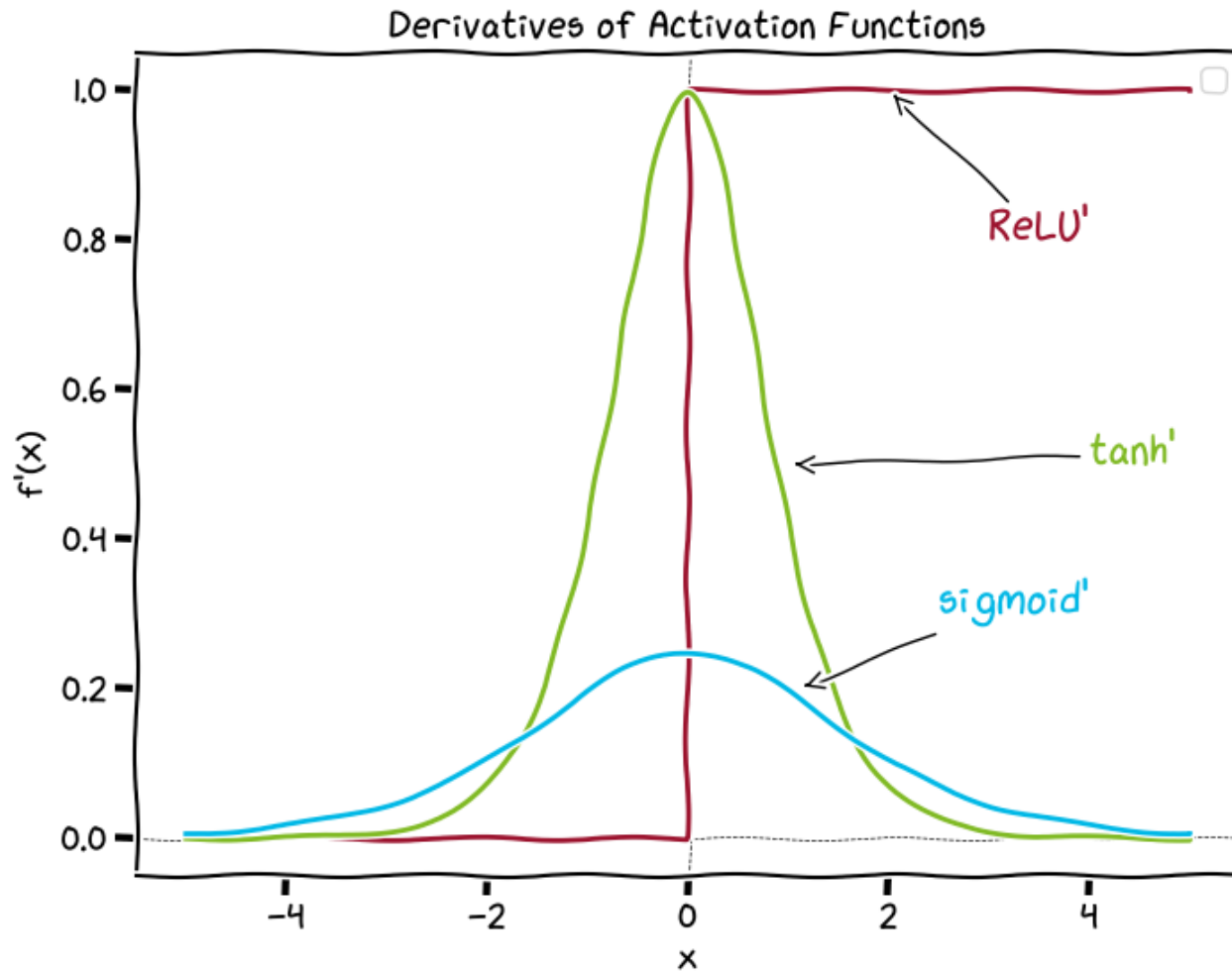
The clouds are in the sky



I moved to germany...

...so I speak german fluently.

1. Trick: Aktivierungsfunktion



Nutzt man z.B. ReLU verhindert das Verkleinern von f' für $x > 0$

2. Trick: Initialisierung

Initialisiere die **Gewichtsmatrix**
mit der Identitätsmatrix

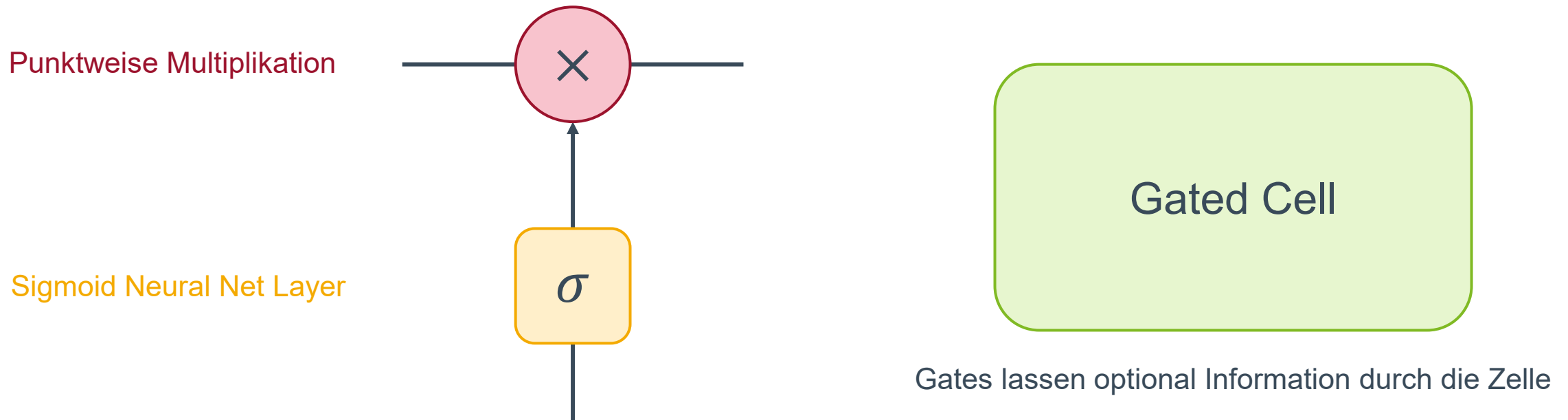
Initialisiere die **Biase** mit 0

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}$$

Das hilft zu verhindern, dass die Gewichte auf 0 schrumpfen

3. Trick: Architektur

Idee: Benutze **Gates** um selektiv Informationen **innerhalb einer Zelle hinzuzufügen** oder zu **löschen**.



Long Short-Term Memory (LSTM)

Sepp Hochreiter, Jürgen Schmidhuber; Long Short-Term Memory. *Neural Comput* 1997; 9 (8): 1735–1780. doi: <https://doi.org/10.1162/neco.1997.9.8.1735>

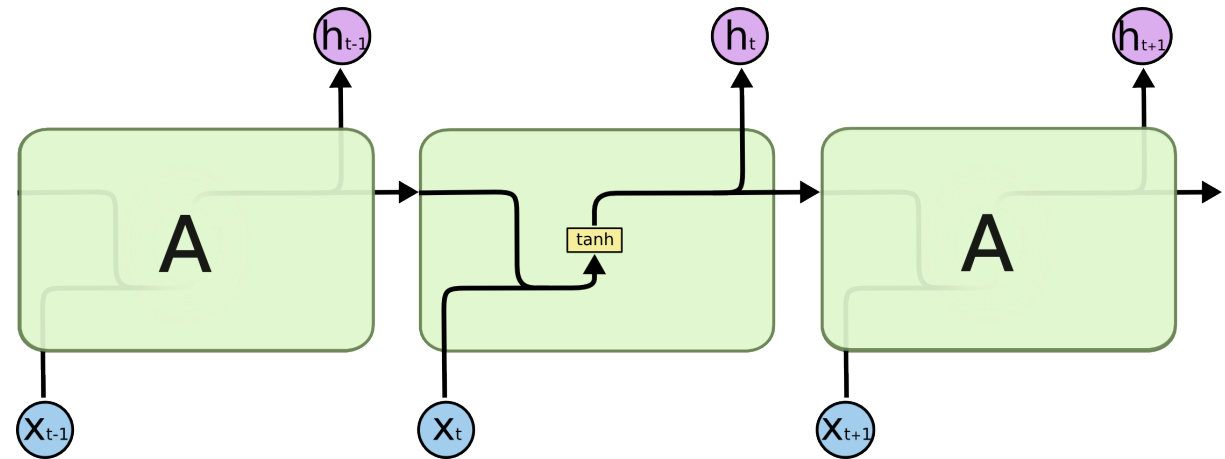
Vanilla RNN vs. LSTM

- Long Short-Term Memory-Netzwerke (LSTM) sind eine spezielle Art von RNN
- LSTMs sind explizit darauf ausgelegt, das Problem der Langzeitabhängigkeit zu vermeiden. Das Erinnern von Informationen über lange Zeiträume ist praktisch ihr Standardverhalten, nicht etwas, das sie mühsam lernen müssen!

$$h_t = f_W(x_t, h_{t-1})$$

↓

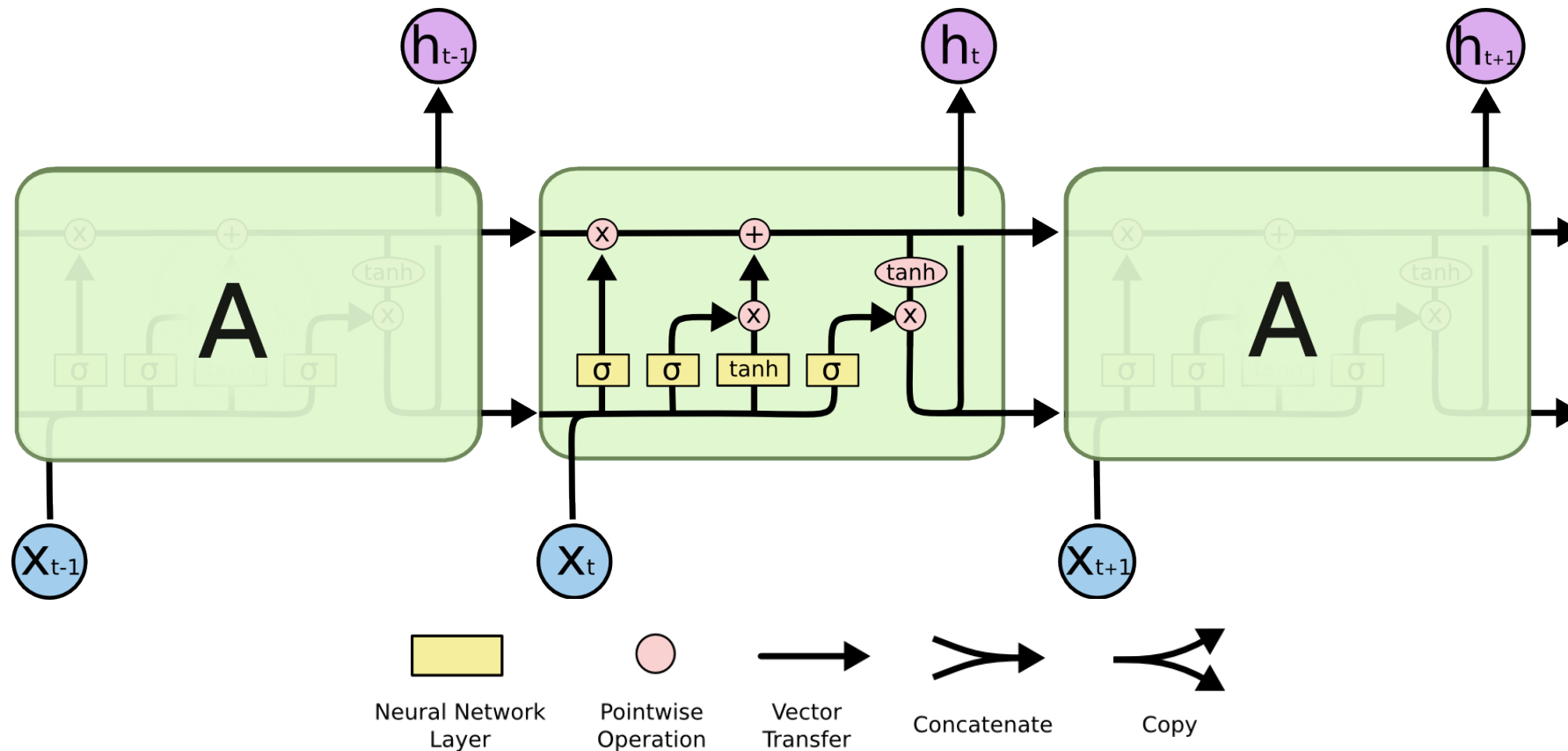
$$h_t = \tanh(W_{hh}^T h_{t-1} + W_{xh}^T x_t)$$
$$y_t = W_{hy}^T h_t$$



Das Wiederholungsmodul in einem Vanilla-RNN enthält nur eine Schicht.

LSTM-Architektur

LSTMs haben ebenfalls diese kettenartige Struktur, aber das Wiederholungsmodul hat eine andere Struktur. Anstelle einer einzigen neuronalen Netzwerkschicht gibt es vier, die auf ganz bestimmte Weise interagieren.

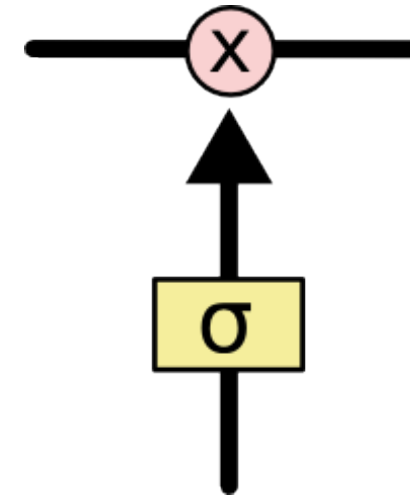
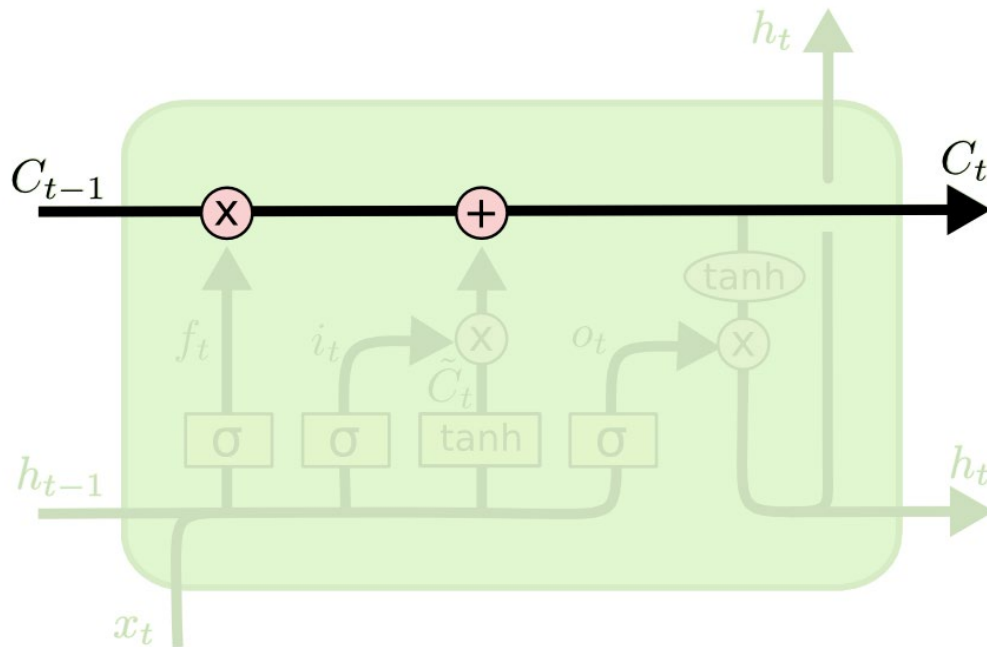


Grafiken: [Understanding LSTM Networks – colah's blog](https://colah.github.io/posts/Understanding-LSTMs/)

LSTM-Architektur

Grundidee:

ein zusätzliches „Förderband“ (Cell State) wird verwendet, um Information zu transportieren.

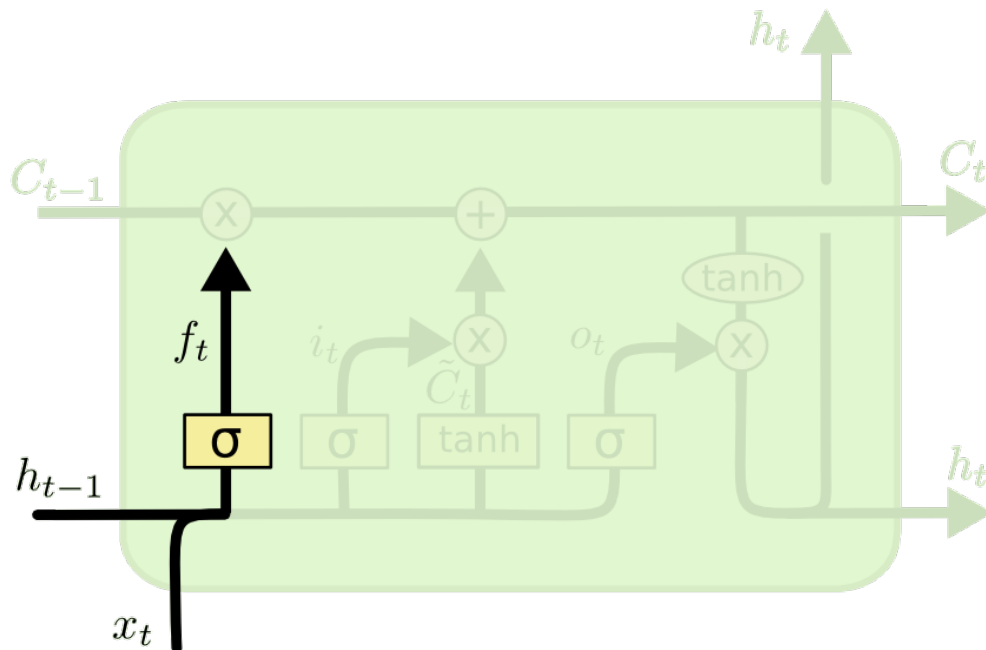


LSTM kann Information hinzufügen oder entfernen durch **Gates**

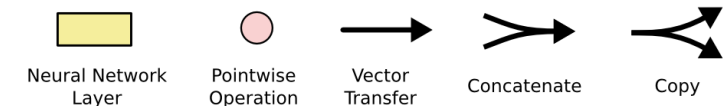
Reguliert durch Sigmoid (produziert Werte zwischen 0-1)

LSTM: Forget Gate

- **Forget:** Welche Informationen sollen behalten werden, welche nicht?
- Sigmoid gibt Zahlen zwischen 0-1 zurück.
Ein Wert von Null bedeutet "nichts durchlassen", während ein Wert von Eins bedeutet "alles durchlassen!,,
- Es betrachtet h_{t-1} und x_t und gibt für jede Zahl im Zellzustand C_{t-1} eine Zahl zwischen 0 und 1 aus

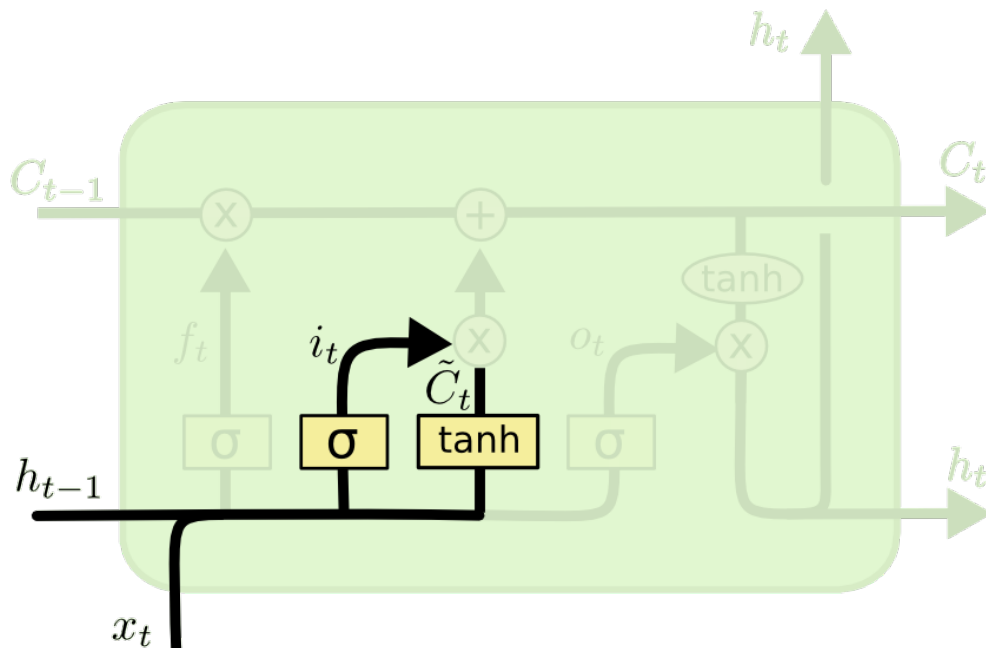


$$f_t = \sigma(W_f \cdot [x_t, h_{t-1}] + b_f)$$



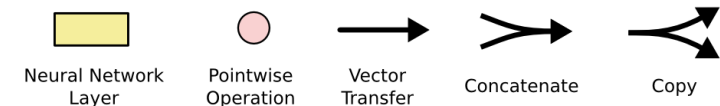
LSTM: Store Gate

- **Store:** Welche neuen Informationen sollen im Zellzustand gespeichert werden?
- Zunächst entscheidet eine Sigmoid-Schicht, die so genannte **Input-Gate-Schicht**, welche Werte wir aktualisieren werden.
- Eine *tanh*-Schicht erzeugt einen Vektor neuer Kandidaten. Die Kombination dieser beiden Schichten führt zu einer Aktualisierung des Zustands.



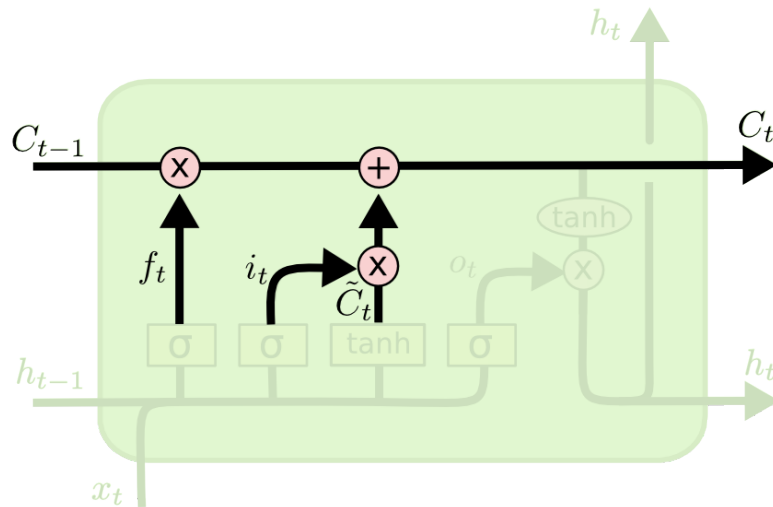
$$i_t = \sigma(W_i \cdot [x_t, h_{t-1}] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

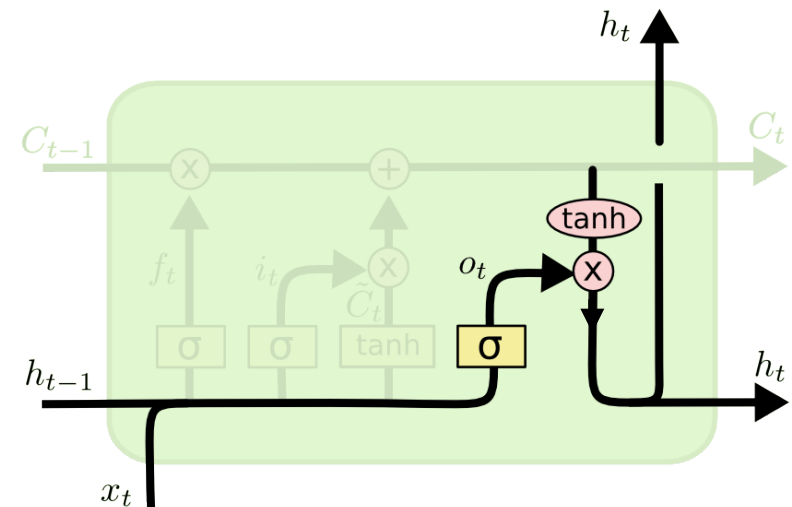


LSTM: Update und Output

Update: In diesem Schritt wird der alte Cell-State durch elementweise Operationen aktualisiert



Output: Schließlich müssen wir entscheiden, welche Werte ausgegeben werden sollen.



Der Zustand der Zelle wird an $\tanh$ übergeben, um Werte zwischen -1 und 1 zu erhalten und mit dem Ausgang des Sigmoid-Gates multipliziert, so dass wir nur die Teile der Informationen ausgeben, für die wir uns entschieden haben.

LSTM: Zusammengefasst

Kernidee:

- Spezielle Art von RNN, entwickelt zur Bewältigung langfristiger Abhängigkeiten.
- Nutzt ein "Gedächtnisband" (Cell State) zur Speicherung und Weitergabe von Informationen über lange Zeiträume.

Architektur:

- Besteht aus vier interagierenden Schichten innerhalb des rekurrenten Moduls.
- Verwendet "Gates" zur selektiven Steuerung des Informationsflusses:
 - **Forget Gate:**
Entscheidet, welche Informationen aus dem Cell State verworfen werden sollen.
 - **Input Gate:**
Entscheidet, welche neuen Informationen dem Cell State hinzugefügt werden sollen.
 - **Output Gate:**
Entscheidet, welche Informationen aus dem Cell State ausgegeben werden sollen.

Weitere Infos: [Implementierung: Long Short-Term Memory \(LSTM\)](#)

LSTM: Beispiel Pytorch

```
1 class LSTMModel(nn.Module):
2     def __init__(self, input_size, hidden_size, num_layers, output_size):
3         super(LSTMModel, self).__init__()
4         self.hidden_size = hidden_size
5         self.num_layers = num_layers
6
7         # LSTM Layer
8         self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
9
10        # Fully connected layer
11        self.fc = nn.Linear(hidden_size, output_size)
12
13    def forward(self, x):
14        # Initialisiere hidden state und cell state
15        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
16        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
17
18        # Forward propagation durch LSTM
19        out, _ = self.lstm(x, (h0, c0))
20
21        # Dekodiere den hidden state der letzten Zeitschritte
22        out = self.fc(out[:, -1, :])
23        return out
```

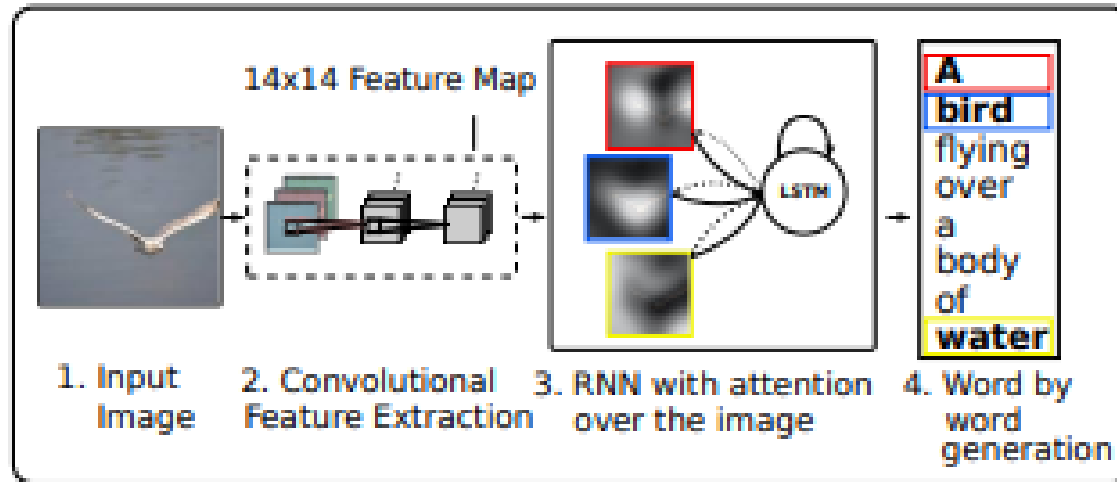
LSTM: Beispiel Pytorch

```
1 def train_model(model, train_loader, num_epochs, learning_rate):
2     # Loss und Optimizer
3     criterion = nn.MSELoss()
4     optimizer = optim.Adam(model.parameters(), lr=learning_rate)
5
6     # Training loop
7     for epoch in range(num_epochs):
8         for i, (inputs, targets) in enumerate(train_loader):
9             # Forward pass
10            outputs = model(inputs)
11            loss = criterion(outputs, targets)
12
13            # Backward pass und Optimierung
14            optimizer.zero_grad()
15            loss.backward()
16            optimizer.step()
17
18            if (i+1) % 100 == 0:
19                print(f'Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{len(train_loader)}],
20                      Loss: {loss.item():.4f}')
```

Applikationen für RNN

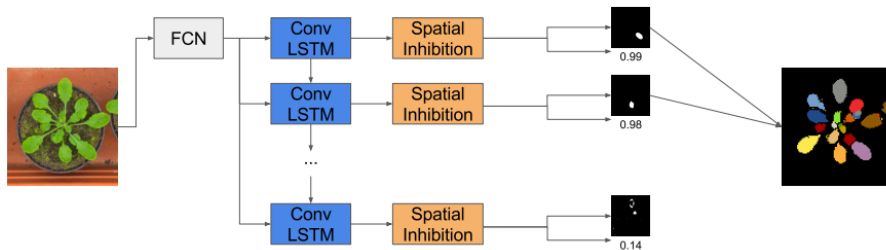
RNN für Computer Vision

Image Captioning



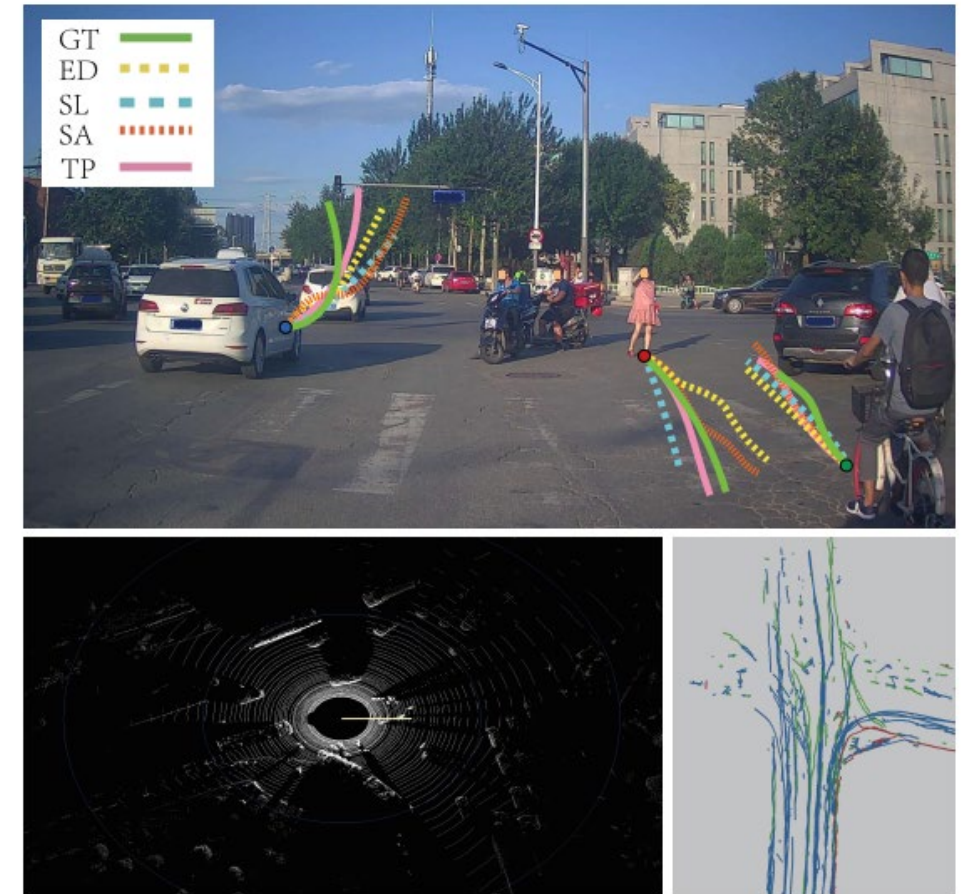
[Show, Attend and Tell: Neural Image Caption Generation with Visual Attention \(arxiv.org\)](#)

LSTM Based Image Segmentation



[Recurrent Instance Segmentation \(arxiv.org\)](#)

LSTM Based Trajectory Prediction



[TrafficPredict: Trajectory Prediction for Heterogeneous Traffic-Agents \(arxiv.org\)](#)

RNN für Sequenzen

Sheet Music Generation

Neural Rhapsody

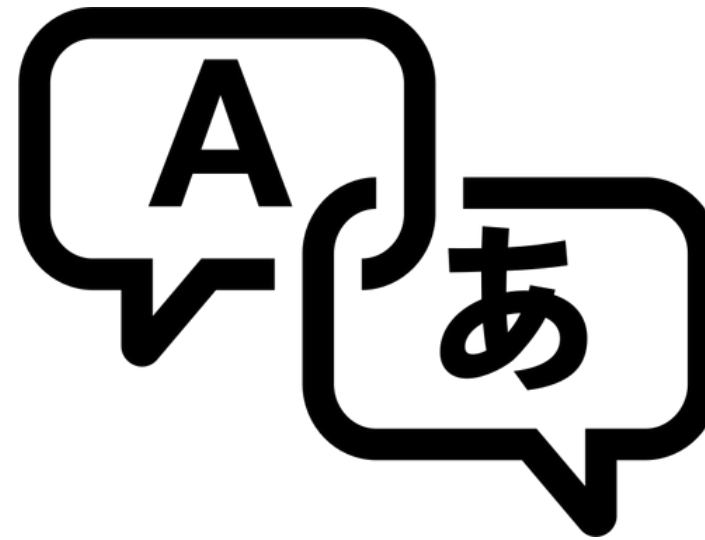
maia (2019)

inspired by Wolfgang Mozart (1756-1791)



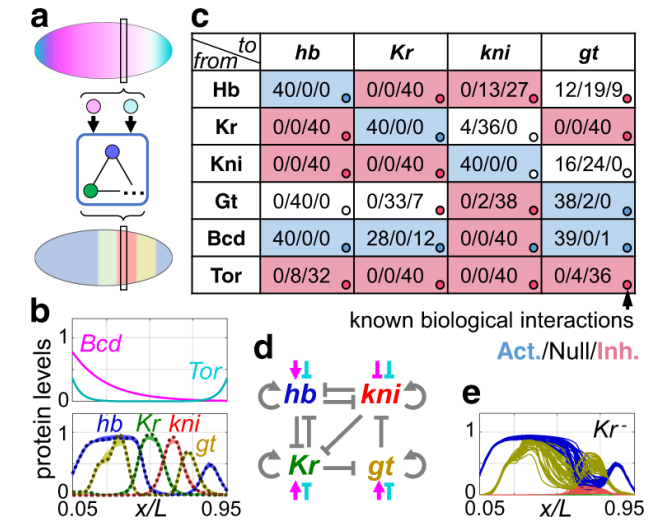
[An Improved RNN-LSTM based Novel Approach for Sheet Music Generation](#)

Text Translation



[RNN based machine translation and transliteration for Twitter data](#)

Biologie



[Finding gene network topologies for given biological function with recurrent neural network](#)